

Table & Feature Evolution Across the Data Pipeline

Online Retail Sales Analytics | Technical-to-Business Data Dictionary

Purpose of This Document

This document explains how the dataset changes as it moves through the pipeline. It focuses on the tables and features created at each layer: which fields are retained, which are cleaned or renamed, which new features are engineered, and why those changes are useful.

Pipeline Overview

Layer	Main table(s)	What changes	Result
Staging	staging_online_retail	Raw UCI fields are loaded with minimal transformation.	Source-like transaction table
Clean / ETL	cleaned_online_retail	Text is standardised, exact duplicates are removed, transactions are classified, quality issues are flagged.	Trusted transaction records
Analytics	analytics_online_retail	Financial measures, time features and analytical flags are engineered.	Business-ready transaction table
Star Schema	fact_sales + dimensions	Descriptive attributes are separated into reusable dimensions and transactions are linked by keys.	Reporting-ready dimensional model
Business Analytics	Queries/views of star schema	Measures are aggregated into KPIs, trends and rankings.	Decision-support outputs

1. Staging Layer — Preserve the Source

The staging table is deliberately close to the source dataset. Its purpose is to provide a controlled landing point before business rules are applied. Apart from a generated row identifier and import timestamp, the original retail fields are preserved.

staging_online_retail

Feature	Origin / transformation	Meaning	Why it exists
staging_id	Generated on import	Unique row identifier in the staging table	Supports traceability through later layers
invoice_no	Source field	Invoice / transaction identifier	Groups line items into invoices and later helps classify cancellations
stock_code	Source field	Product code	Identifies the product
description	Source field	Product description	Human-readable product information
quantity	Source field	Number of units recorded	Used to distinguish sales from negative-quantity

			activity and calculate value
invoice_date	Source field	Transaction date and time	Supports chronological and time-based analysis
unit_price	Source field	Price per unit	Used with quantity to calculate transaction value
customer_id	Source field	Customer identifier where available	Supports customer-level analysis
country	Source field	Country attached to the transaction	Supports geographic analysis
imported_at	Generated on import	When the row entered the database	Operational lineage / audit metadata

2. Clean / ETL Layer — Standardise, Deduplicate and Classify

The clean layer keeps the core transaction fields but makes their representation consistent. Blank text values are converted to NULL, text is trimmed, exact duplicate rows are removed, and two important interpretive features are added: transaction_status and data_quality_flag.

cleaned_online_retail — retained and cleaned fields

Feature	Origin / transformation	Meaning	Why it exists
cleaned_id	Generated in clean layer	Unique cleaned-row identifier	Identifies the clean record
source_staging_id	Copied from staging_id	Link back to the original imported row	Preserves data lineage
invoice_no	TRIM + blank → NULL	Standardised invoice identifier	Consistent matching and classification
stock_code	TRIM + blank → NULL	Standardised product code	Consistent product identification
product_description	description renamed; TRIM + blank → NULL	Clean product description	Clearer naming and consistent missing-value handling
quantity	Retained	Recorded unit quantity	Core transaction measure
invoice_date	Retained	Transaction timestamp	Core time attribute
unit_price	Retained	Recorded price per unit	Core financial input
customer_id	TRIM + blank → NULL	Customer identifier where available	Preserves anonymous sales while enabling known-customer analysis
country	TRIM + blank → NULL	Transaction country	Supports geographic analysis
cleaned_at	Generated	Clean-layer load timestamp	Audit metadata

New ETL features

Feature	Origin / transformation	Meaning	Why it exists
transaction_status	Rule-based classification	Cancelled / Adjustment / Completed / Review	Separates different business events instead of treating every row as a normal sale
data_quality_flag	Rule-based quality check	Valid or a specific issue such as missing customer, zero price, etc.	Makes remaining data-quality limitations visible rather than silently deleting records

Exact duplicate handling

A temporary duplicate_rank is calculated across invoice number, stock code, product description, quantity, invoice date, unit price, customer ID and country. Only duplicate_rank = 1 is loaded into the clean table. duplicate_rank is therefore transformation logic, not a permanent feature of cleaned_online_retail.

3. Analytics Layer — Engineer Business-Ready Features

The analytics layer retains the cleaned transaction context and adds features designed for measurement, filtering and aggregation. This is the point where operational fields such as quantity and unit price become explicit business measures such as completed revenue and net revenue.

Lineage and retained transaction fields

Feature	Origin / transformation	Meaning	Why it exists
analytics_id	Generated	Unique analytics-row identifier	Identifies the analytics record
source_cleaned_id	From cleaned_id	Link to clean-layer record	Lineage
source_staging_id	Carried forward	Link to staging record	End-to-end lineage
invoice_no, stock_code, product_description	Carried forward	Transaction and product context	Keeps the analytics row understandable
quantity, unit_price	Carried forward	Base numeric inputs	Used in financial measures
invoice_date	Carried forward	Full transaction timestamp	Retains exact event time
customer_id, country	Carried forward	Customer and geographic context	Supports segmentation
transaction_status, data_quality_flag	Carried forward	Business classification and quality context	Allows measures to respect earlier business rules

Financial features

Feature	Origin / transformation	Meaning	Why it exists
transaction_value	quantity × unit_price	Signed value of the line item	Base monetary value before business-specific separation
completed_revenue	transaction_value only when status = Completed	Revenue from completed transactions	Separates normal sales performance
cancelled_value	ABS(quantity × unit_price) when Cancelled	Magnitude of cancelled transaction value	Allows cancellations to be reported as a positive magnitude
adjustment_value	ABS(quantity × unit_price) when Adjustment	Magnitude of adjustment activity	Keeps adjustments visible without mixing them into completed revenue
net_revenue	quantity × unit_price for Completed and Cancelled; otherwise 0	Net effect of included sales and cancellations	Supports a revenue view that retains cancellation effects

Time features

Feature	Origin / transformation	Meaning	Why it exists
invoice_date_only	DATE(invoice_date)	Calendar date without time	Links transactions to dim_date
sales_year	YEAR(invoice_date)	Year	Annual analysis
sales_quarter	QUARTER(invoice_date)	Quarter	Quarterly analysis
sales_month	MONTH(invoice_date)	Month number	Monthly grouping and ordering
sales_month_name	MONTHNAME(invoice_date)	Month label	Readable reporting
sales_week	WEEK(invoice_date, 3)	Week number	Weekly analysis
sales_day	DAY(invoice_date)	Day of month	Daily analysis
sales_day_name	DAYNAME(invoice_date)	Weekday name	Day-of-week patterns
sales_hour	HOUR(invoice_date)	Hour of transaction	Intraday analysis
is_weekend	Derived from day of week	Weekend indicator	Simple weekday vs weekend segmentation

Analytical flags

Feature	Origin / transformation	Meaning	Why it exists
is_completed	1 when Completed,	Completed transaction	Fast filtering and counting

	otherwise 0	indicator	
is_cancelled	1 when Cancelled, otherwise 0	Cancellation indicator	Cancellation analysis
is_adjustment	1 when Adjustment, otherwise 0	Adjustment indicator	Adjustment analysis
is_review	1 when Review, otherwise 0	Review indicator	Isolates unresolved/unusual transactions
has_customer_id	1 when customer_id is present	Customer-identification availability	Separates identified vs anonymous transactions
created_at	Generated	Analytics-layer load timestamp	Audit metadata

4. Star Schema — Separate Measures from Descriptive Context

The star schema changes the structure more than the meaning. Instead of repeating date, product, customer and country attributes on every analytical row, reusable dimension tables hold that descriptive context. fact_sales keeps the invoice-line-item grain and connects each transaction to those dimensions using keys.

dim_date

Feature	Origin / transformation	Meaning	Why it exists
date_key	YYYYMMDD-style key from invoice_date_only	Dimension key	Connects fact_sales to a calendar date
full_date	From invoice_date_only	Actual calendar date	Readable date
calendar_year	Derived	Year	Time analysis
calendar_quarter	Derived	Quarter	Time analysis
calendar_month	Derived	Month number	Time analysis
month_name	Derived	Month name	Readable reporting
week_of_year	Derived	Week number	Weekly analysis
day_of_month	Derived	Day number	Daily analysis
day_name	Derived	Weekday name	Weekday patterns
day_of_week	Derived	Numeric weekday	Ordering / grouping
is_weekend	Derived	Weekend indicator	Weekend segmentation

dim_product

Feature	Origin / transformation	Meaning	Why it exists
product_key	Generated surrogate key	Product dimension key	Links products to fact_sales
stock_code	From analytics layer	Business product identifier	Stable product reference
product_description	Representative description per stock_code	Human-readable product label	Resolves multiple descriptions by keeping the most frequently used description

dim_customer

Feature	Origin / transformation	Meaning	Why it exists
customer_key	Generated surrogate key	Customer dimension key	Links customers to fact_sales
customer_id	Distinct customer IDs; UNKNOWN record added	Business customer identifier	Supports customer-level aggregation while retaining missing-customer transactions

dim_country

Feature	Origin / transformation	Meaning	Why it exists
country_key	Generated surrogate key	Country dimension key	Links countries to fact_sales
country_name	Distinct country values;	Geographic label	Supports country-level

	Unknown record added		reporting
--	----------------------	--	-----------

fact_sales

Feature	Origin / transformation	Meaning	Why it exists
sales_key	Generated	Unique fact-row identifier	Identifies one fact row
source_analytics_id	From analytics_id	Lineage to analytics layer	Traceability
date_key, product_key, customer_key, country_key	Mapped to dimensions	Foreign keys	Connect each sale to reusable descriptive context
invoice_no	Retained	Invoice identifier	Invoice-level counting and analysis
invoice_datetime	Renamed from invoice_date	Exact transaction timestamp	Preserves transaction timing
sales_hour	Retained analytical feature	Hour of transaction	Intraday analysis without re-derivation
quantity, unit_price	Retained	Base numeric measures	Volume and price analysis
transaction_status, data_quality_flag	Retained	Classification and quality context	Allows transparent filtering
transaction_value	Retained	Signed line value	Financial analysis
completed_revenue	Retained	Completed-sales revenue	Sales KPI
cancelled_value	Retained	Cancellation magnitude	Cancellation KPI
adjustment_value	Retained	Adjustment magnitude	Adjustment monitoring
net_revenue	Retained	Net included revenue effect	Net performance KPI
is_completed, is_cancelled, is_adjustment, is_review	Retained	Status flags	Fast filtering and aggregation
has_customer_id	Retained	Customer-availability flag	Customer coverage analysis
created_at	Generated	Fact-table load timestamp	Audit metadata

5. What Changes from Analytics Layer to Star Schema?

This transition is mainly a structural normalisation for reporting. The financial meaning created in the analytics layer is preserved in fact_sales, while repeated descriptive attributes are moved into dimensions.

Analytics-layer feature	Star-schema destination	Change
invoice_date_only + derived date attributes	dim_date	Moved into a reusable calendar dimension
stock_code + product_description	dim_product	Moved into product dimension; representative description selected
customer_id	dim_customer	Distinct customers stored once; missing values map to UNKNOWN
country	dim_country	Distinct countries stored once; missing values map to Unknown
financial measures and status flags	fact_sales	Retained at invoice-line-item grain
invoice_date	fact_sales.invoice_datetime	Retained with clearer fact-table naming
analytics_id	fact_sales.source_analytics_id	Preserved for lineage

6. Feature Evolution Summary

The feature set becomes progressively more purposeful. Source features are first preserved, then standardised and classified, then enriched with business measures, and finally reorganised into a model designed for reporting.

- Staging: What did the source provide?
- Clean / ETL: Is the record consistent, unique and interpretable?
- Analytics: What does the transaction contribute financially, when did it occur, and how should it be filtered?
- Star Schema: Which attributes describe the transaction, and which measures belong at the sales-event level?
- Business Analytics: How can those measures be aggregated into performance KPIs and business insights?

7. Important Modelling Notes

- The grain of fact_sales is one invoice line item (stock code) per invoice.
- Missing customer IDs are intentionally retained because the sales event can still be useful even when the customer is unknown.
- UNKNOWN / Unknown dimension records prevent missing descriptive references from automatically removing valid fact rows.
- Revenue definitions are established in the analytics layer and then carried into fact_sales; the star schema should not redefine their business meaning.
- The SQL source defines dim_date.calendar_quarter, although the provided INSERT statement aliases QUARTER(invoice_date_only) as calendar_month. This document describes the intended feature based on the table definition; the SQL alias should be reviewed before final production use.